

Developing an Intelligent Chatbot

Abstract

This report presents the design and implementation of an intelligent chatbot system capable of assisting users across three core railway service functions. The system integrates natural language processing, a knowledge base, a reasoning engine, and a machine learning model to deliver these functionalities. First, the chatbot helps passengers find the cheapest available train tickets by collecting journey details and querying the National Rail API to retrieve and compare available fares. Second, it predicts the arrival time of delayed trains on the South Western Railway route between Weymouth and London Waterloo using a Random Forest model trained on historical operational data. Third, it provides an expert system interface for train operational staff to receive guidance on handling service contingencies through a rule-driven knowledge base. The system was developed in Python, with NLTK and spaCy powering the natural language understanding component. Evaluation through live demonstration and testing confirms that the chatbot handles all three scenarios with coherent conversational flow and reliable prediction performance, demonstrating the practical application of NLP, ensemble learning, and rule-based reasoning in a real-world railway context.

1. Introduction

The field of artificial intelligence has undergone a remarkable transformation in recent decades. What was once considered experimental is now an essential component of modern platforms and services across virtually every industry. The rapid advancement of artificial intelligence has enabled the development of intelligent chatbots that can interact with users naturally and efficiently. Platforms such as ChatGPT, Claude, and Gemini have further brought AI into the mainstream, allowing its users the facility to converse with a system that responds like a human yet possesses the knowledge and reasoning capability of an expert. These systems are being adopted across industries to automate customer service and reduce operational costs. The railway sector is no exception, with several train operating companies already deploying chatbot solutions to assist passengers with journey planning, ticket booking, and real-time service updates. The report documents the design and development of an intelligent chatbot system that assists users in finding the cheapest train tickets across the UK, predicts the revised arrival time of a delayed train, and acts as an expert advisor for train operational staff in case of emergencies.

1.1 Background and Motivation

Chatbots are machine agents that engage in communication with humans using natural language. Alan Turing in 1950 proposed the Turing Test (“Can machines think?”), and it was at that time that the idea of a chatbot was popularized. The first known chatbot was Eliza, developed in 1966, whose purpose was to act as a psychotherapist returning the user utterances in a question form. Over the time the main focus in chatbot design moved from the perfect imitation of human to the development of helpful tools able to support the users in their tasks with a more natural interaction achieved through the use of natural language. The communication with a chatbot can take place by writing text or with a speech-recognition system. The need for a Railway Enquiry Chatbot arises from the increasing demand for instant, efficient, and personalized assistance in the railway sector, to find cheapest tickets and information about train delays. Traditional methods of obtaining information, such as waiting in long queues or navigating complex websites, can be time-consuming and inconvenient. A chatbot eliminates these challenges by offering 24/7 availability, quick response times, and the ability to handle multiple queries simultaneously. It also provides personalized recommendations based on user preferences, ensuring a seamless and engaging interaction.

1.2 Aim and Objectives

The aim of this project is to build an intelligent chatbot capable of assisting its users in finding the cheapest available train tickets in the UK, providing good customer service by predicting when a delayed train will actually arrive at the destination based on real operational data, and acting as an expert advisor that guides train operational staff through the correct procedures for handling service contingencies and emergencies.

The objectives of the projects are:

1. Cheapest ticket search and booking assistance

- a. To develop a chatbot capable of having conversation with user, collect their journey details such as departure station, destination, travel date and time, return journey preferences etc.

- b. To search for cheapest tickets by integrating with the external train ticket booking services, National Rail, via available APIs to retrieve live ticket pricing and availability that matches the user's requirements.
- c. To identify and present the cheapest available ticket to the user with a direct hyperlink to the relevant booking website.

2. Train-Delay Prediction and Improved Customer Service

- a. To process and structure the provided historical train operational data into a suitable format.
- b. To train, evaluate, and compare multiple predictive models for estimating revised arrival times given a reported delay at an intermediate station.
- c. To integrate the best-trained prediction model into the chatbot so that it can tell the user the estimated real arrival time at their destination.

3. Expert-System to deal with contingencies

- a. To extract relevant rules and procedures from the regulation documents concerning train service contingencies.
- b. To construct a structured knowledge base (KB) encoding these rules, and implement a reasoning engine capable of querying it in response to staff queries.
- c. To provide train operational staff with suggested action plans through the interface.

4. System Architecture and User-Interface

- a. To design and implement a Natural Language Processing and Understanding component capable of interpreting user queries accurately and appropriately.
- b. To design and maintain a supporting database to store train station information, historical operational data, conversation logs, and knowledge base entries.
- c. To deliver a seamless user experience across all three tasks through a chatbot interface, ensuring the chatbot provides relevant and concise responses and information to the user.

1.3 Difficulties and Risks

1. Accessing Live Train Ticket Data for task 1: Getting data from The National Rail requires registration and may have access delays, and web scraping alternatives carry legal and technical risks that could prevent live ticket retrieval during demonstration
2. Building an Accurate Delay Prediction model for task 2: The quality of the delay prediction model is heavily dependent on the completeness and consistency of the provided historical data.
3. Building the Knowledge Base for Contingency Management: Extracting rules from complex regulatory documents is difficult and challenging as incorrectly encoded rules could produce wrong advice to the user.
4. Natural Language Understanding: It is not always sure users will ask the questions in a predicted manner. They may phrase the same question in many different ways, make spelling mistakes, or use slang and abbreviations. The chatbot may misunderstand user intent and give irrelevant or incorrect responses.

1.4 Work Plan

Tasks	Main task	Secondary task	Work
Task - 1	Ticket Finder, UI, Integration	Testing ticket module	1. Collect journey details from user 2. Connect to API / data source 3. Retrieve ticket options 4. Display cheapest ticket 5. Build simple text-based UI
Task - 2	NLP, Conversation Manager, Architecture	Integration support	1. Detect user intent (ticket, delay, contingency) 2. Extract entities (stations, dates, delay minutes) 3. Manage conversation states 4. Route queries to the correct module 5. Handle fallback responses
Task - 3	Delay Model, Prediction Expert System, Database	Model support integration	1. Preprocess historical train data 2. Train delay prediction model 3. Integrate chatbot prediction into 4. Extract rules from contingency documents 5. Build rule-based expert system

Table 1: Work Plan

2. Related Work

In the last few years, railway organizations have been using more and more chatbot technologies to improve customer experience and provide real-time assistance. The effectiveness of conversational AI in the transportation industry is demonstrated through various implementations. Northern Trains system deploys WhatsApp-based chatbot on Azure Cognitive Service. This conversational chatbot can help you find out live train schedules, service updates and taxi details. This system depicts the advantage of integration of messaging platforms with AI to provide instant and user-friendly services. Another interesting implementation is Penny, developed by TransPennine Express. Penny is a web-based digital assistant for passengers, helping them book tickets, check timetables and get live updates about their journeys. This system decreases the dependence on manual customer support and ensures efficient query resolution. The French Railways, the SNCF, has deployed chatbots on popular social media platforms Facebook Messenger and Twitter. Such chatbots offer features like buying tickets, managing bookings and processing customers' claims. This approach shows how the integration of chatbots with well-known platforms increases accessibility and user engagement. The Central Japan Railway Company has launched an AI-powered, browser-based chatbot in Japan. The main function of this chatbot is to help users to find connecting trains and plan complicated routes by train. It is based on accuracy and efficiency for multi-step journey planning.

3. Methods, tools and Framework

3.1 Methods

Task 1 uses a custom NLU method to detect user intent and extract key entities such as stations, dates, times, and railcards. It applies rule-based NLP using regular expressions to interpret natural language inputs. The system also uses API-based journey planning to fetch UK train information and session-based dialogue management to maintain context across multiple user turns.

For Task 2 train delay prediction, three prediction methods were trained and compared; kNN, XGBoost, and Random Forest. The project uses Random Forest as the primary prediction model as it performed the best. Supervised regression was used to predict additional delay in minutes.

For Task 3 uses a combination of rule-based and retrieval-based methods to generate disruption guidance. The system applies rule-based slot filling to extract key incident details and uses guided multi-turn dialogue to ensure all required information is collected before producing advice. Exact SQL matching retrieves structured contingency plans, while semantic retrieval identifies relevant operational rules when wording varies. Finally, role-specific formatting ensures the output is tailored to passengers, station staff, signallers, service controllers, or all roles as needed.

3.2 Languages, Packages, Tools

The project is built in Python 3.10. Scikit-learn provides the implementation for the random forest and K-Nearest Neighbor algorithms, while XGBoost handles gradient boosted tree modelling. All trained models are saved using joblib for serialization. NumPy is used to construct the feature arrays passed to those models. FastAPI serves as the web framework for the API on the backend, with Uvicorn acting as the ASGI server to run it.

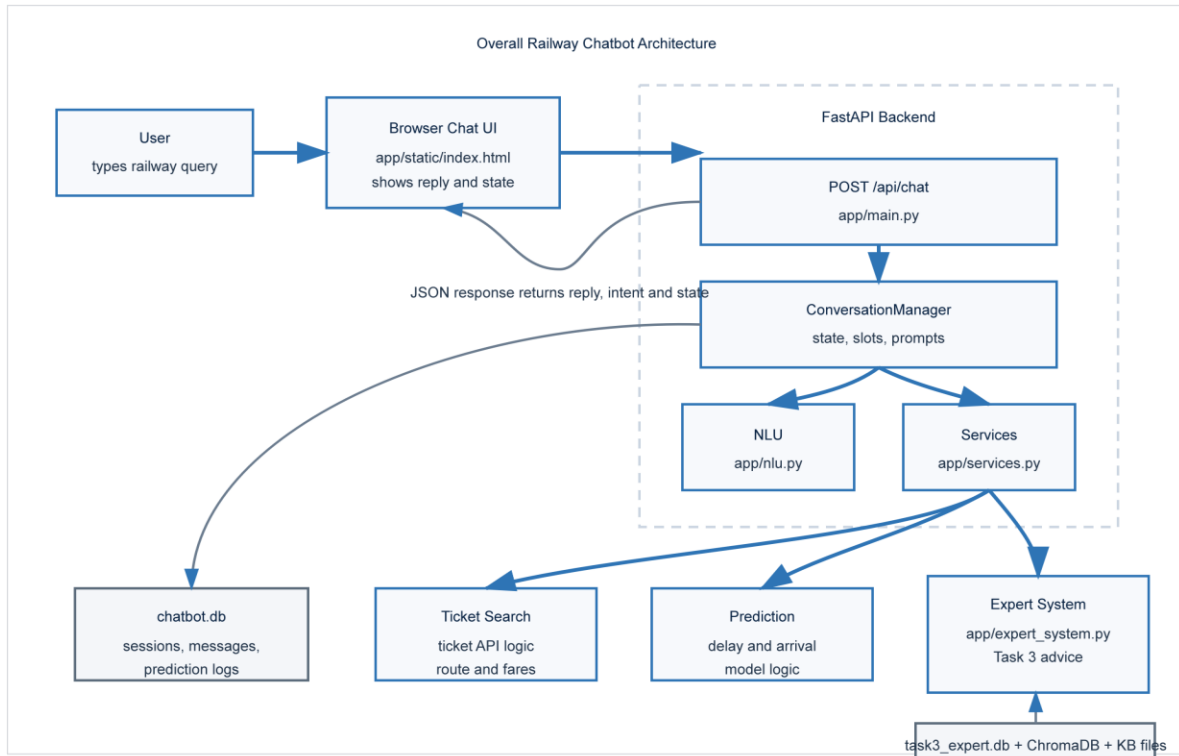
4. Design of the Chatbot

This section describes the design of the railway chatbot as a complete system. The chatbot combines three main functions in one interface: finding train ticket information, predicting delay or arrival outcomes, and giving expert-system guidance for railway disruption scenarios. The design focuses on keeping the user conversation simple while allowing the backend to route each request to the correct service.

The system is built around a browser-based chatbot interface and a FastAPI backend. The backend manages conversation state, extracts meaning from user messages, stores session data, and calls the ticket, prediction, or expert-system modules when enough information has been collected.

4.1 The Architecture of the Chatbot

The chatbot follows a client-server architecture. The user interacts with the web interface in `app/static/index.html`, and each message is sent to the FastAPI endpoint `POST /api/chat` in `app/main.py`. The request contains the session ID and the user's message, while the response returns the chatbot reply, detected intent, dialogue state, collected slots, and any prediction output.



The main control layer is the ConversationManager in app/conversation.py. This module decides whether the user is asking for ticket search, delay prediction, arrival-time prediction, contingency advice, or an unknown query. It stores the conversation state, checks which slots are missing, asks follow-up questions, and only completes the task when the required information has been gathered.

The service layer in app/services.py separates the dialogue logic from the operational tasks. Ticket requests are passed to the ticket API logic, delay and arrival requests are passed to prediction functions, and disruption requests are passed to the expert system. This separation makes the chatbot easier to test because each part can be checked independently before being used in the full conversation flow.

The architecture also includes local storage. The chatbot.db SQLite database stores sessions, messages, and prediction logs, while task3_expert.db stores structured railway disruption knowledge for the expert system. ChromaDB is used alongside this database to provide semantic retrieval when a natural-language disruption query does not exactly match a structured rule.

4.2 User Interface

The user interface is a simple browser chatbot implemented in app/static/index.html. It provides a message area, an input box, a send action, and a status line showing the current intent and state

returned by the backend. This design keeps the interface easy to use because the user can type ordinary language rather than filling in a long form.

The interface communicates with the backend through the `/api/chat` endpoint. When the user sends a message, JavaScript sends the message as JSON and then displays the reply returned by FastAPI. This means the frontend does not need to contain the reasoning logic itself; it only presents the conversation and sends user input to the backend.

The guided conversation design is important for usability. If the user does not give all required details in one message, the chatbot asks a clear follow-up question. For example, ticket search may require origin, destination, travel time, and ticket type, while disruption advice may require the affected stations, blockage type, and requested guidance category.

4.3 NLU

The natural language understanding component is implemented mainly in `app/nlu.py`. It uses deterministic rule-based methods to identify the user's intent and extract useful entities from the message. The supported intents are `ticket_search`, `delay_prediction`, `arrival_time_prediction`, `contingency_advice`, and `unknown`.

Intent detection is based on keywords and phrases related to each task. For example, words such as `ticket`, `fare`, `cheapest`, and `journey` indicate ticket search, while `delay` and `arrival` terms route the message to prediction workflows. Disruption phrases such as `line blockage`, `station disruption`, `ticket acceptance`, or `contingency advice` route the message to the expert-system task.

Entity extraction fills the slots required by each workflow. For tickets, the chatbot extracts details such as origin, destination, travel date and time, and return preference. For prediction, it extracts stations, travel context, and delay minutes. For disruption advice, it extracts the event type, affected location, blockage type, station name, and requested advice category.

The NLU design is intentionally controlled rather than fully generative. This is suitable for the coursework system because the chatbot has a defined set of tasks and must collect reliable inputs before calling APIs, prediction logic, or expert-system reasoning.

4.4 Knowledgebase

The chatbot uses more than one knowledge source. General conversation state and logs are stored in `chatbot.db`, while the Task 3 expert knowledge is stored in `task3_expert.db`. The expert database contains structured records for line blockage plans and station disruption plans, allowing the system to match user inputs to relevant railway contingency guidance.

The Task 3 knowledge base is also supported by extracted files in `knowledge_base_task3`. These files contain text from railway contingency and station disruption planning documents. During

ingestion, the information is converted into structured database records and semantic rule cards so that the chatbot can retrieve advice in a practical form.

ChromaDB provides the vector-based retrieval layer. It is used when exact SQL matching is not enough or when the user's wording is closer to a natural-language description than a database key. This combination of SQLite and ChromaDB gives the chatbot both precise structured lookup and flexible semantic retrieval.

The knowledge-base design helps keep user responses focused. Instead of displaying raw source files, database paths, or retrieval scores, the chatbot formats the selected knowledge into readable advice for the user.

4.5 Inferring Engine

The inferring engine is the part of the chatbot that decides what answer should be returned after the user's intent and slots are known. In the overall chatbot, inference happens through the conversation manager and the service layer. These components decide whether to search for tickets, run delay logic, run arrival prediction, or query the expert system.

For Task 1, the inference process checks whether the required ticket slots have been collected, then calls the ticket search service. The service attempts to use the live ticket API logic and returns a structured ticket response containing route, travel time, ticket type, price or availability status, provider, and booking link.

For Task 2, the inference process validates the prediction inputs before producing an answer. Delay prediction uses route and station context to estimate a risk level and predicted delay, while arrival-time prediction uses the trained model artefact and service context to estimate the final delay and expected arrival time. If required inputs or model data are missing, the chatbot returns a safe unavailable response instead of giving an unreliable prediction.

For Task 3, the expert-system inference engine is implemented mainly in `app/expert_system.py`. It first tries structured SQL matching against `task3_expert.db` using the collected incident slots. If useful, it also searches ChromaDB through `app/task3_rag.py` and merges relevant semantic matches. The final output is then converted into focused operational guidance for the requested disruption scenario.

Overall, the inferring engine makes the chatbot more than a simple question-answer interface. It remembers the conversation, identifies the task, collects missing facts, selects the correct backend logic, and returns a response that is suitable for the user's current railway problem.

4.6 Task -1: Ticket Search and Conversational System

Task 1 focuses on developing the chatbot's ability to understand natural language queries and assist users in finding UK train tickets. The system combines rule-based natural language understanding, slot-based dialogue management, and structured ticket retrieval logic to guide the user through the

process of specifying their journey details. Once all required information is collected, the chatbot retrieves real or mock ticket options and presents them in a clear, user-friendly format.

4.6.1. System Workflow

The chatbot works as a slot-filling dialogue. It does not assume the user will provide all information in the first message. The ConversationManager loads the session, detects intent, extracts entities, updates known slots, checks the first missing required slot, and either asks a focused question or completes the ticket-search flow.

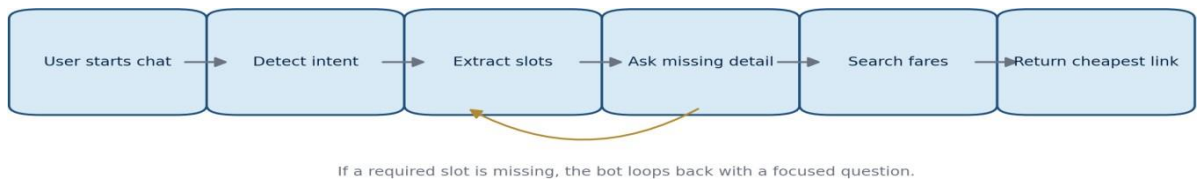


Figure 1. Dialogue workflow for collecting journey details and returning a fare result.

For ticket search, the required slots are origin, destination, outbound_datetime and return_preference. The implementation also checks whether outbound_datetime contains both a date expression and a time expression. If the user says only '15 July', the bot asks for the time; if the user says only '8am', it asks for the date. This keeps the conversation natural while still ensuring the API request is complete.

4.6.2 Implementation Architecture

The application is structured as a small FastAPI service with clear separation between web delivery, conversation management, language understanding, persistence, and ticket integration. This separation makes it easier to replace the ticket supplier, improve the NLU layer, or add new operators without rewriting the whole chatbot.

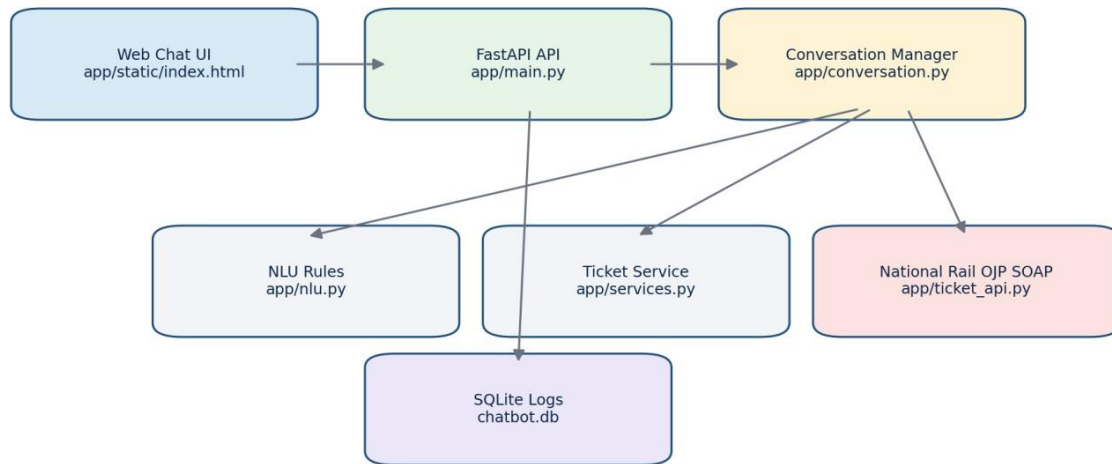


Figure 2. Main components in the implemented chatbot architecture.

4.6.3 Intent Detection

The first step in Task 1 is determining what the user wants to do. The system uses a rule-based intent detector that analyses the user's message for keywords and patterns associated with ticket searching. For example, phrases such as "book a train", "find me a ticket", "travel from X to Y" are mapped to the ticket_search intent. This ensures that the chatbot can correctly identify when the user is requesting journey information and activate the appropriate dialogue flow.

4.6.4 Entity Extraction and Slot Filling

Once the intent is identified, the system extracts key journey details from the user's message. These include:

- **Origin station**
- **Destination station**
- **Travel date**
- **Travel time**
- **Return preference** (single or return)
- **Railcard usage**

The extraction process uses regular expressions and rule-based parsing to identify dates, times, and station names. If the user provides incomplete information, the system asks targeted follow-up questions. For example, if the user says “I want to go to London tomorrow”, the chatbot detects the date and destination but asks for the departure time.

This slot-based approach ensures that all required information is collected before a ticket search is performed.

4.6.5 Railcard Handling

The system includes specialised logic for handling railcards. If the user indicates they have a railcard, the chatbot asks them to specify the type (e.g., 16–25 Railcard, Senior Railcard). If the user says they do not have one, the railcard slot is set to "none". This prevents ambiguity and ensures that ticket pricing can be calculated correctly

4.6.6 Station Validation

To prevent invalid inputs, the system validates all station names against a list of recognised UK railway stations. If the user enters a non-UK or unrecognised station, the chatbot responds with an error message and asks the user to provide a valid UK station. This ensures that only correct station names are passed to the backend ticket search logic.

4.6.7 Conversation Manager

A dedicated conversation manager controls the dialogue flow throughout Task 1. It:

- Tracks which slots have been filled
- Determines the next question to ask
- Maintains session state across multiple turns
- Handles corrections and re-entries
- Ensures the conversation progresses logically

For example, if the user provides a return journey but does not specify the return date, the manager detects the missing slot and prompts the user accordingly. This structured approach ensures a smooth and consistent user experience.

4.6.8 Ticket Retrieval Logic

Once all required slots are collected, the system attempts to retrieve ticket information. Two pathways are used:

a. Live API Search

If the National Rail API is available, the system sends a structured request containing:

- Origin
- Destination
- Outbound date and time
- Return date and time (if applicable)
- Railcard

The API returns real journey options including departure times, arrival times, platforms, ticket types, and prices.

b. Mock Ticket Generator

If the API is unavailable or returns an error, the system uses a fallback mock generator. This produces realistic ticket details such as:

- Departure and arrival times
- Journey duration
- Platform numbers
- Ticket type
- Price
- Booking link

This ensures the chatbot always returns a usable result, even without external data.

4.6.9 Response Formatting

The final ticket information is formatted into a clear, readable summary. For single journeys, the chatbot presents:

- Origin - Destination
- Departure and arrival times
- Duration
- Ticket type

- Price
- Booking link

For return journeys, both outward and return legs are displayed along with the total price.

This ensures that the user receives a complete and easy-to-understand ticket summary.

4.6.10 Summary

Task 1 establishes the core conversational capabilities of the chatbot. It integrates intent detection, entity extraction, slot filling, validation, and ticket retrieval into a cohesive system that can reliably support users in finding UK train tickets. This foundation enables the more advanced predictive features developed in Task 2 and ensures a smooth, consistent user experience across the entire chatbot system.

Railway Chatbot

Intent: ticket_search | State: completed

Hello, how can I help you find ?

I want to book a ticket

What station are you travelling from?

Norwich

What is your destination station?

London

Type a message...

Send

Fig 3: Conversation between the user and Chatbot about the travel

4.7 Delay Prediction Models

4.7.1 Dataset Description

The dataset provided consists of historical train performance records from South Western Railway (SWR), covering services operating between London Waterloo (WAT) and Weymouth (WEY), in both directions, from 2022 to 2025. The dataset contains approximately 577,187 records for the WAT to WEY direction and 346,862 records for the WEY to WAT direction, indicating an imbalance in the number of observations between the two routes. Each record in the dataset relates to a single train at a station and includes the necessary information such as planned arrival time, actual arrival time, planned departure time, actual departure time, station location code, date of service, and a unique journey identifier.

Prior to modeling, several preprocessing steps were applied. Time values originally stored in HH:MM format were converted into minutes past midnight to enable numerical processing. Records containing missing values in any required field were removed to ensure dataset consistency. In addition, outlier filtering was done by keeping only the records where departure delay fell within the range of -30 to + 200 minutes. The lower bound of -30 minutes was chosen to exclude the implausible early departures likely caused by data entry errors, while the upper bound of +200 minutes was set to remove extreme outliers that represent exceptional cases rather than typical cases, as it could skew the model towards rare events.

4.7.2 Feature Engineering

Rather than predicting raw arrival times, the work predicts how much a train's delay will change between two stations. For each journey, the source destination pairs were constructed by joining each station call with all subsequent station calls on the same service. The target variable, `additional_delay`, is the difference between the arrival delay at the destination and the current departure delay. The model is trained to answer, "given that a train is currently X minutes late, will that delay get better or worse by the time it reaches the destination?"

Twelve features were derived for each source-destination pair, as summarized below. Temporal features such as `planned_dep_min`, `day_of_week`, `month`, `is_weekend`, and `is_peak` capture the influence of time of day, day and seasonal patterns on train punctuality. Features such as `station_num`, `journey_progress`, `stations_remaining`, `destination_num`, and `stops_to_dest` tell

where the train currently is and how far it must travel. Finally, `current_delay` provides the model with the delay already happening at the point of prediction.

Feature	Description	Rationale
<code>Planned_dep_min</code>	Planned departure time in minutes since midnight	Time of day affects congestion
<code>hour</code>	Hour of departure	Groups departures by hour to capture general time of day patterns
<code>Day_of_week</code>	Mon-Sun (0-6)	Weekdays and weekends differ in demand
<code>month</code>	Month (1-12)	Captures seasonal and weather effects
<code>Is_weekend</code>	1 is Sat/Sun	Weekend timetables differ
<code>Is_peak</code>	1 if during morning/evening rush hour	Peak hours have more delays
<code>Station_num</code>	Position of current station on the route	Where the train is along the line
<code>Journey_progress</code>	<code>Station_num</code> as a 0-1 ratio	Provides comparable position across both directions
<code>Stations_remaining</code>	Stations left until end of line	How much of the journey remains
<code>Destination_num</code>	Position of passenger's destination	Identifies the target station
<code>Stops_to_dest</code>	Stations between now and destination	How far away the destination is
<code>Current_delay</code>	Current delay in minutes	Late trains tend to stay late

Table 2: Feature descriptions and rationale for inclusion in the predictive model

4.7.3 Models Trained

Three machine learning models were trained and evaluated: Random Forest, k-Nearest Neighbours, and XGBoost. All three models were trained independently on identical dataset for each direction of travel, with the same random seed (42) applied throughout to ensure reproducibility.

For Random Forest, two hyperparameters were tuned: the number of estimators with values of 50, 100, and 200, and the maximum tree depth with values of 5, 10, 20, and None, yielding 12 combinations. The sample fraction was kept to 0.3 as the dataset size was large. For KNN, the number of neighbours, k was evaluated across five values: 3, 5, 10, 15, and 20. As KNN is a distance-based algorithm, all features were standardised using StandardScaler prior to training to ensure no single feature dominated the distance calculation. For XGBoost, the number of estimators (100, 200, 300) and maximum depth (3, 5, 7) were tuned, with the learning rate of 0.1.

The algorithms were trained to produce different model according to the direction, WAT to WEY and WEY to WAT. The figures below show the number of data sample generated during training phase in both directions.

```
=====
DIRECTION: WAT->WEY
=====

Loading 4 file(s)...
Total rows loaded: 577,187
    Generating pairs (sample=30%)...
Training examples after pairing: 1,449,877

*****
Hyperparameter Tuning (70% train / 15% val / 15% test)
*****

Train:      1,015,493 rows (70% of data)
Validation: 216,902 rows (15% of data)
Test:       217,482 rows (15% of data)
```

Figure 4 : Number of data sample during the hyperparameter tuning (WAT to WEY)

```

-----
Final Model (80% train / 20% test)
  n_estimators=300, max_depth=7
-----

Train: 1,159,901 rows | Test: 289,976 rows

```

Figure 4.1: Number of data sample used for training and testing after finding the best hyperparameters (WAT to WEY)

```

=====
DIRECTION: WEY->WAT
=====

Loading 4 file(s)...
Total rows loaded: 346,861
  Generating pairs (sample=30%)...
Training examples after pairing: 880,712

*****
Hyperparameter Tuning (70% train / 15% val / 15% test)
*****

Train:      616,850 rows (70% of data)
Validation: 131,755 rows (15% of data)
Test:       132,107 rows (15% of data)

```

Figure4.2 : Number of data sample during the hyperparameter tuning (WEY to WAT)

```

-----
Final Model (80% train / 20% test)
  n_estimators=300, max_depth=7
-----

Train: 704,569 rows | Test: 176,143 rows

```

Figure4.3 : Number of data sample used for training and testing after finding the best hyperparameters (WEY to WAT)

For all three models, data was partitioned into training (70%), validation (15%), and test (15%) subsets. Grid search was done to get the best values for the hyperparameters. The best hyperparameter configuration was then used to train a final model on an 80% training and 20%

test split. Prior to saving the model, each final model was refitted to maximize the volume of training data available for deployment.

4.7.4 Evaluation

The model performance was assessed on the test set using three complementary metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2). RMSE was selected as it penalizes larger errors more heavily, which is important in a delay prediction context where large mispredictions are more harmful. MAE provides an interpretable average error in minutes, and R^2 indicates how much of the variance in delay the model explains. Three models were evaluated for both directions of travel.

4.7.4.1 XGBoost

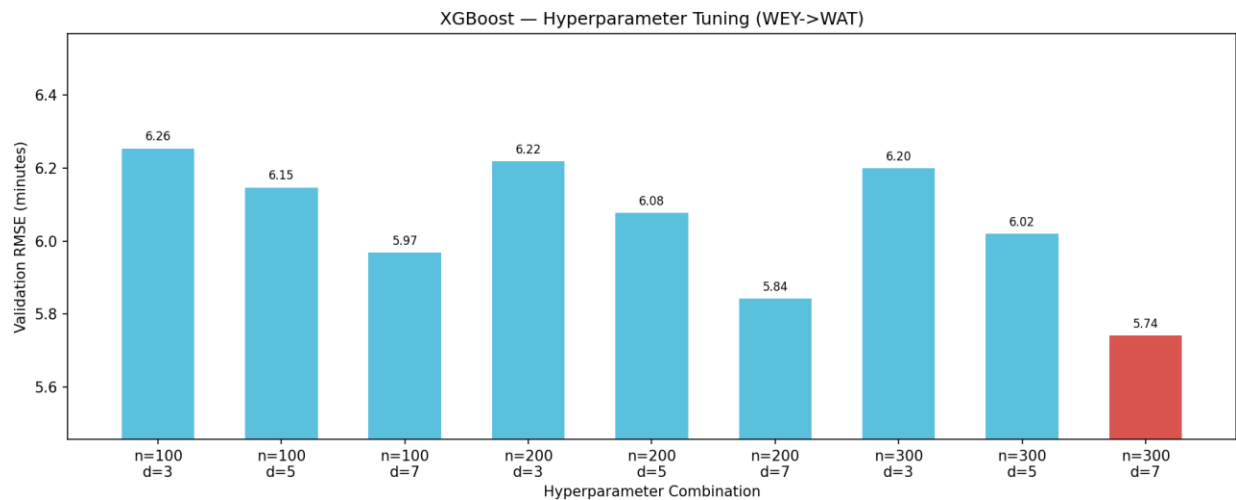


Fig5: Hyperparameter tuning (WEY to WAT)

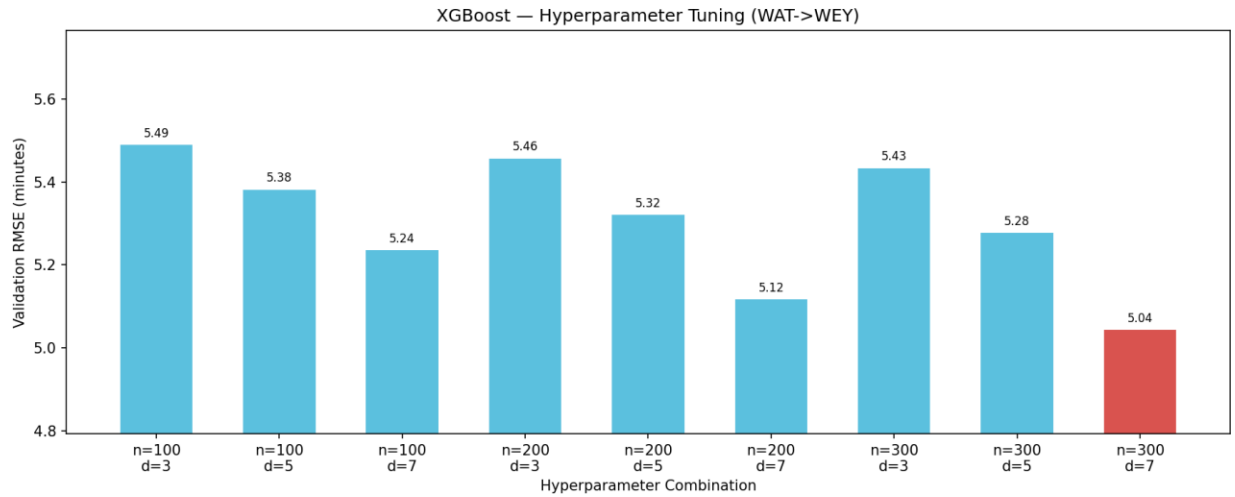


Fig 5.1: Hyperparameter tuning (WAT to WEY)

For both directions, the model with $n=300$ estimators and depth $d=7$ achieved the best performance, with validation RMSE of 5.74 on WEY to WAT and 5.04 on WAT to WEY. Predictions are generally accurate for small delays but underestimate larger ones, a pattern visible in both directions.

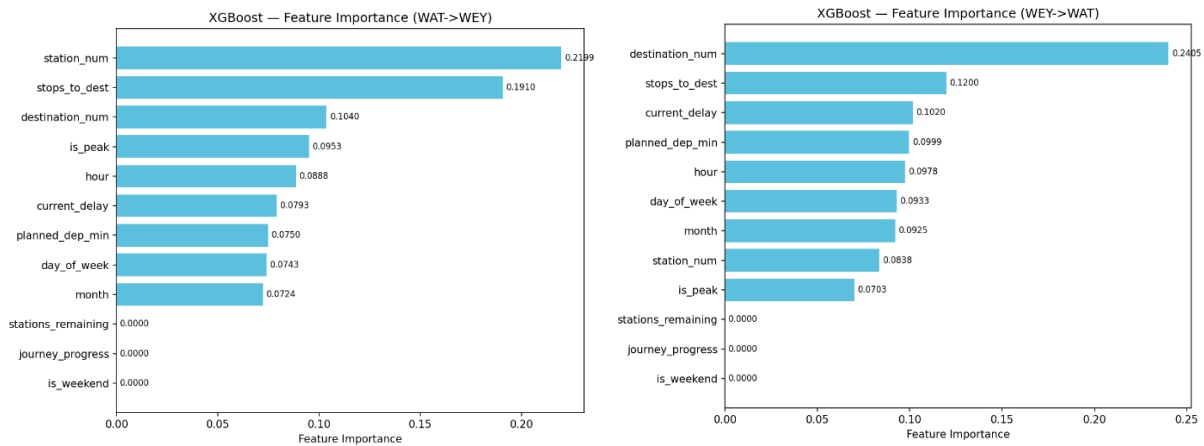


Fig 5.2: Feature Importance XGBoost (both direction)

The most influential features were `destination_num` and `stops_to_dest`, accounting for over 36% of the model's predictive weight for WEY2WAT. For WAT2WEY, the most influential feature was `station_num` which is interesting considering the `station_num` remained on the lower end in feature importance of WEY2WAT. This shows journey position is the primary factor of predictions.

stations_remaining, journey_progress, and is_weekend contribute zero importance in both directions, making them redundant features.

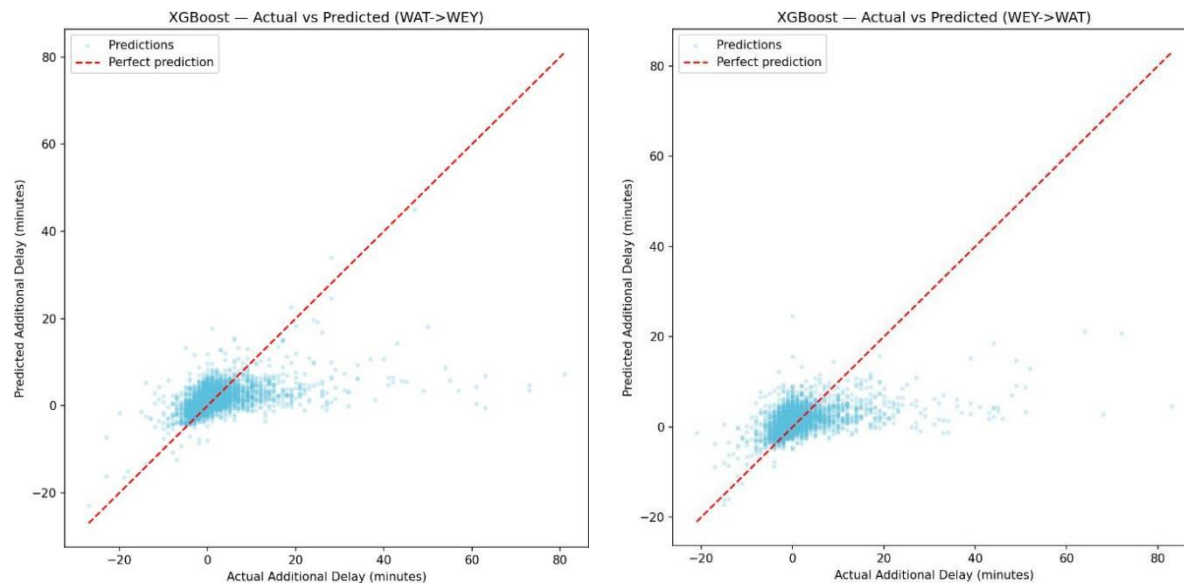


Figure 5.3: Actual vs Predicted Delay in XGBoost (in both direction)

Predictions track well along the perfect prediction line for small to moderate delays. The model underestimates larger delays, with predictions rarely exceeding ~40 minutes regardless of actual delay size.

4.7.4.2 Random Forest

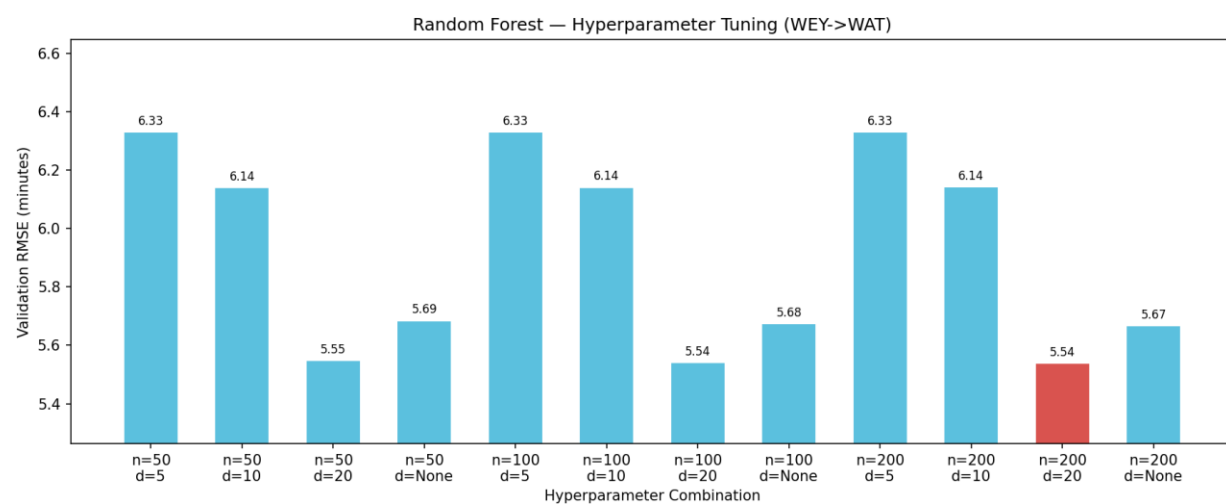


Fig 5.4: Hyperparameter tuning in Random Forest (WEY2WAT)

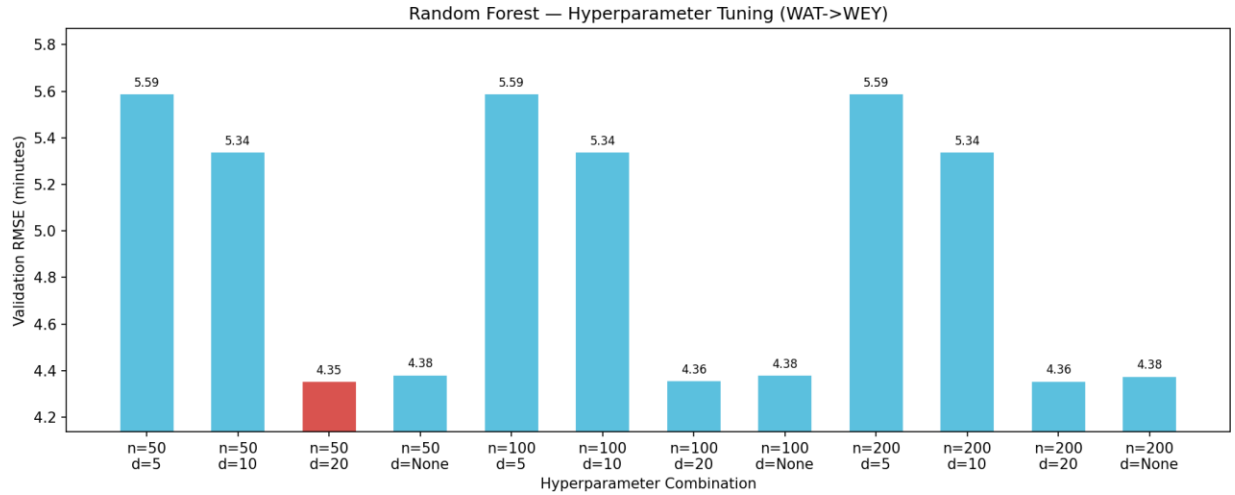


Fig 5.5: Hyperparameter tuning in Random Forest (WAT2WEY)

For direction from WEY to WAT, the best configuration used $n = 200$ estimators and $\text{max_depth} = 20$, achieving a test RMSE of 5.54 minutes on WEY to WAT. For direction from WAT to WEY, the best configuration was $n = 50$ and $\text{max_depth} = 20$ giving RMSE of 4.35.

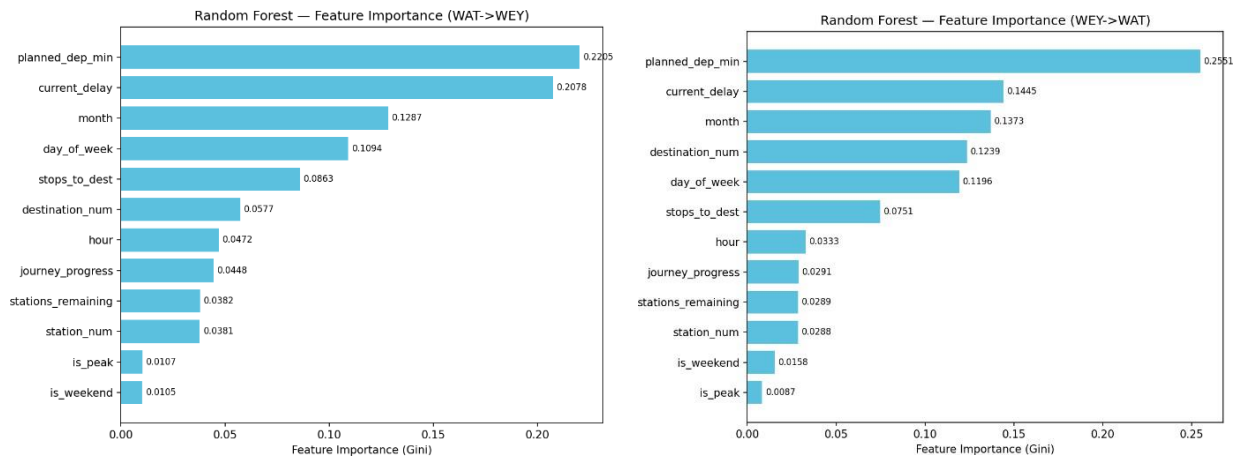


Figure 5.6: Feature Importance in Random Forest in both direction

Across both directions, `planned_dep_min` is the most influential feature which indicates that the time of day a train departs is the strongest predictor of how delay evolves. Notably, the features such as `stops_to_dest`, `destination_num` rank lower here compared to XGBoost, where they dominated. This suggests Random Forest is capturing a broader mix of signals rather than relying heavily on journey position alone. `is_peak` and `is_weekend` contribute very little in both directions which is somewhat surprising given peak hours are associated with congestion. This may be

because their information is already captured implicitly by planned_dep_min and day_of_week, making them redundant once those features are present.

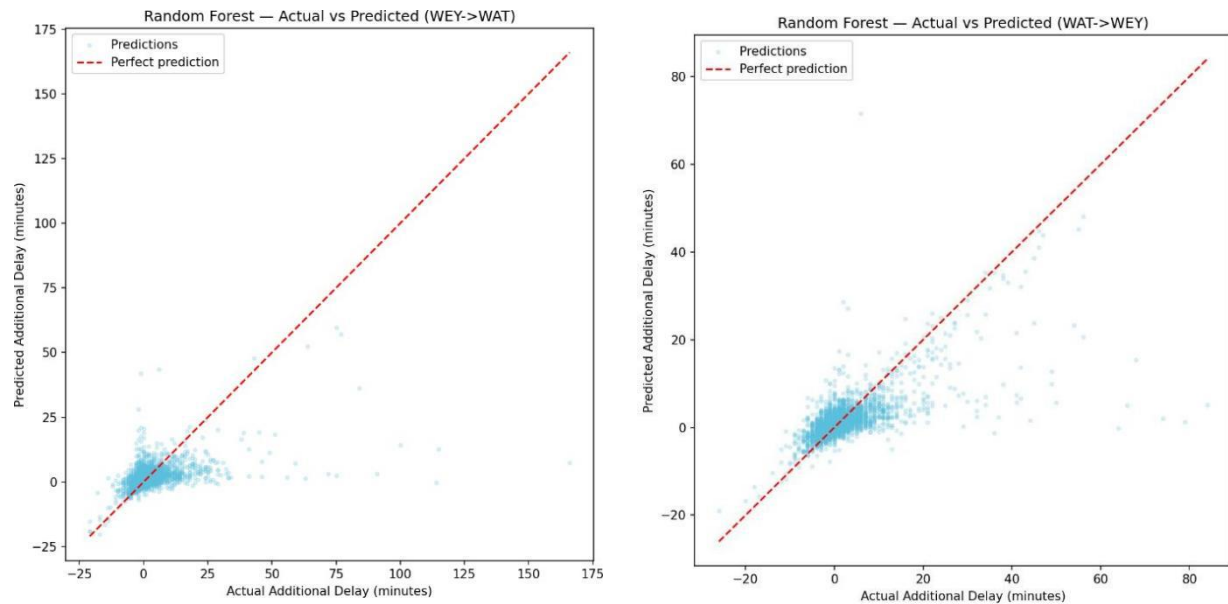


Figure 5.7: Actual vs Predicted Additional Delay in Random Forest (in both direction)

Random Forest shows a tight cluster around the prediction line, particularly for small-to-moderate delays. The models handle the 0 to 25 minutes range well in both directions.

4.7.4.3 kNN

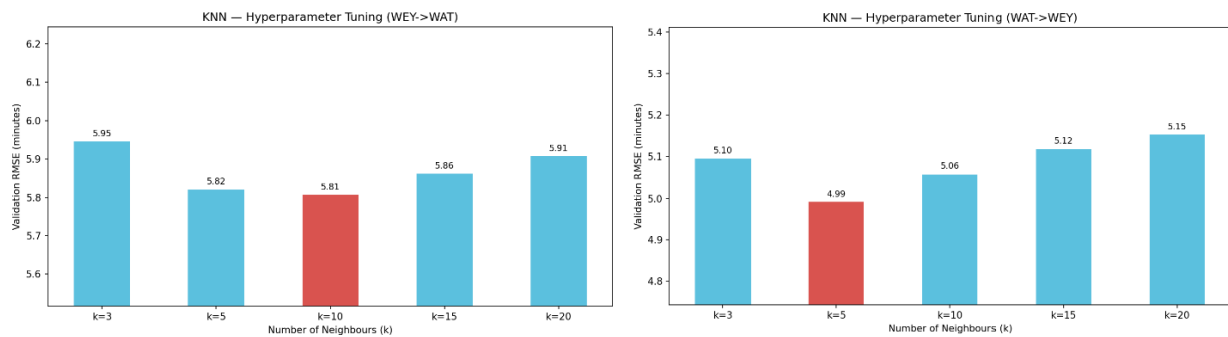


Fig 5.8: Hyperparameter tuning in KNN

For WEY to WAT, k= 10, and for WAT to WEY, k= 5 was selected as the best number of neighbours based on validation RMSE.

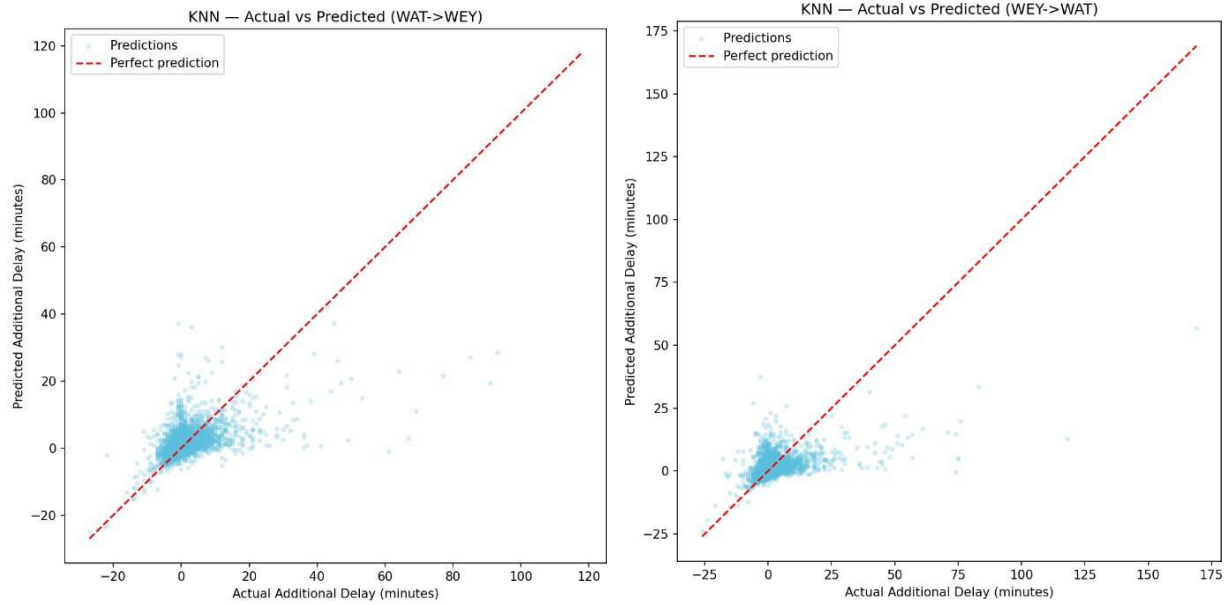


Fig 5.9: Actual vs Predicted Delay in minutes in KNN (both directions)

Both directions show the same pattern: most predictions stay close to zero no matter how large the actual delay is. The model works fairly well for small delays but strongly underestimates medium and large delays. Predictions rarely go above about 20 minutes, even when actual delays are between 120 and 175 minutes. This shows a weakness of KNN, as it averages nearby values together, causing predictions to stay close to the mean and making it poor at predicting very large delays.

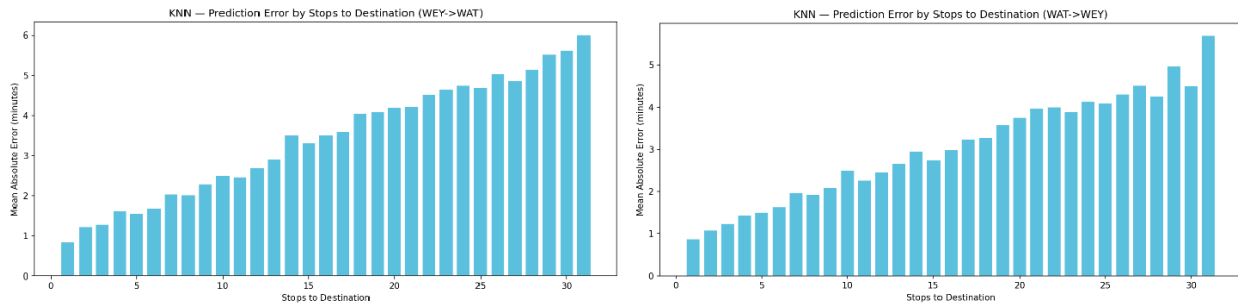


Fig 5.10: Prediction Error by Stops to Destination

Error increases consistently as stops to destination increases, in both directions. This is expected as the further away the destination, the more uncertainty grows. Short-journey predictions (0 to 5 stops) are relatively reliable with MAE under approximately 1.5 minutes, but beyond 20 stops the error climbs to 5 to 6 minutes.

4.7.5 Comparison

The table below summarizes the test-set performance of all three models for both directions of travel. Random Forest consistently achieves the lowest RMSE and highest R^2 in both directions, outperforming KNN and XGBoost across all metrics.

Model	WAT to WEY (RMSE)	WAT to WEY (MAE)	WAT to WEY (R^2)	WEY to WAT (RMSE)	WEY to WAT (MAE)	WEY to WAT (R^2)
Random Forest	4.220	2.125	0.482	5.626	2.553	0.277
KNN	4.896	2.504	0.303	5.813	2.764	0.228
XGBoost	4.999	2.527	0.274	5.819	2.692	0.227

Table 3: Test-set performance metrics for all three models across both travel directions.

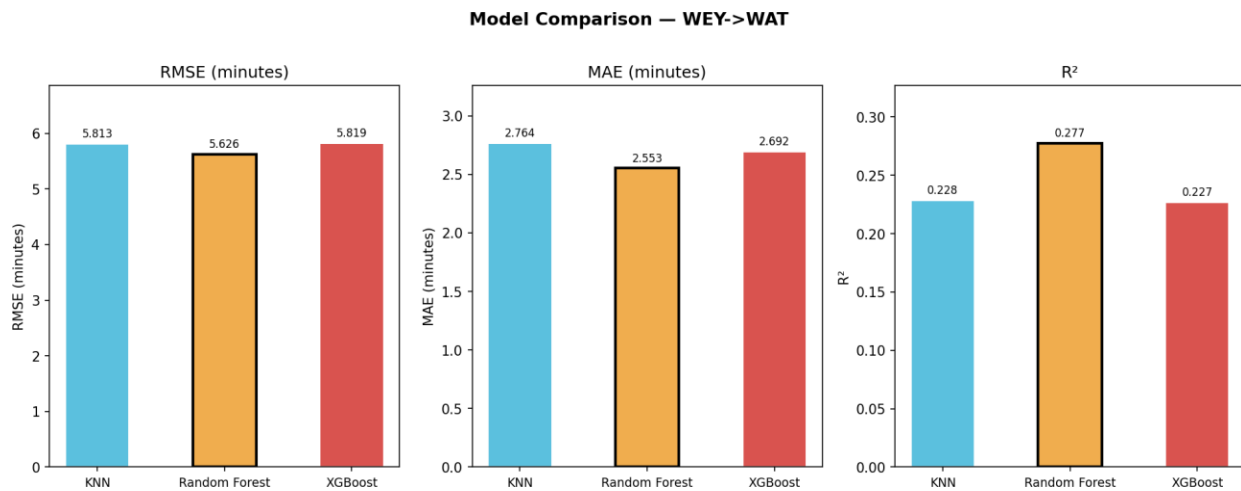


Fig 6: Models Comparison (WEY to WAT)

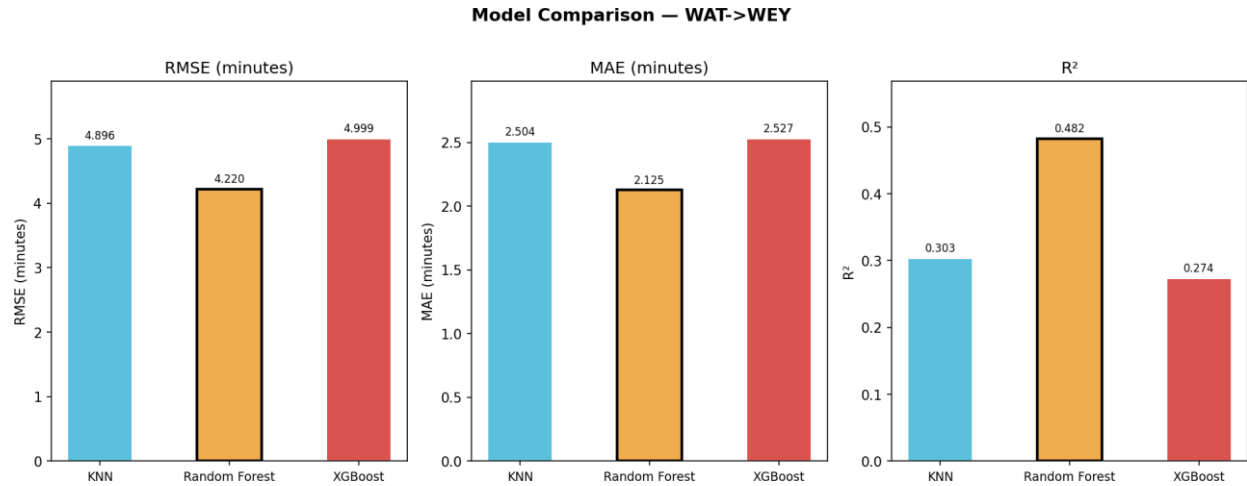


Fig6.1: Models Comparison (WAT to WEY)

Delay category matrices shown below were produced for each model and direction, mapping predictions into five categories: Early (<-5 min), On Time (-5 to 5 min), Slight Delay (5-15 min), Moderate Delay (15-30 min), and Severe Delay (>30 min). The high on-time accuracy across all models (above 90%) reflects a class imbalance in the training data, where near-zero delays cases significantly outnumber moderate and severe delay cases, causing all models to be biased towards predicting small delays. For severe delays (>30 min), XGBoost correctly classifies only 3.3% of cases on the WEY to WAT route, while Random Forest achieves 11.0% on the same route and 30.8% on the WAT to WEY route. KNN also shows the weak performance on severe delays, correctly identifying only 4.8% on WEY to WAT and 14.4% on WAT to WEY. The models are very good at predicting trains that are roughly on time, but poor at predicting large delays which could be because the dataset contains far more normal journeys than severely delayed ones.

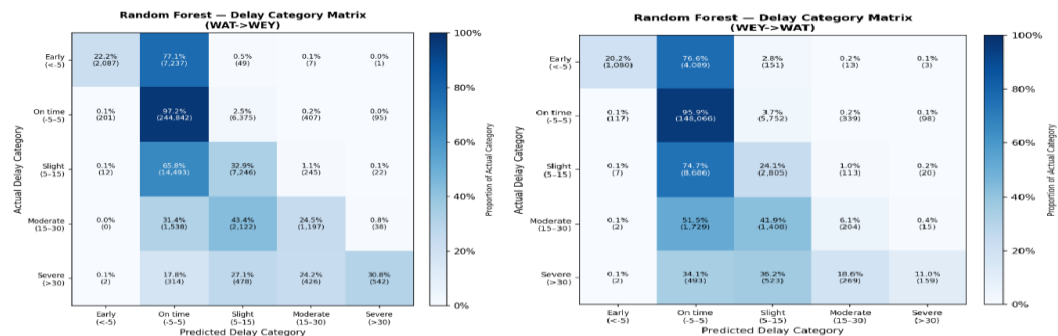


Fig6.2: Delay Category Matrix (Random Forest)

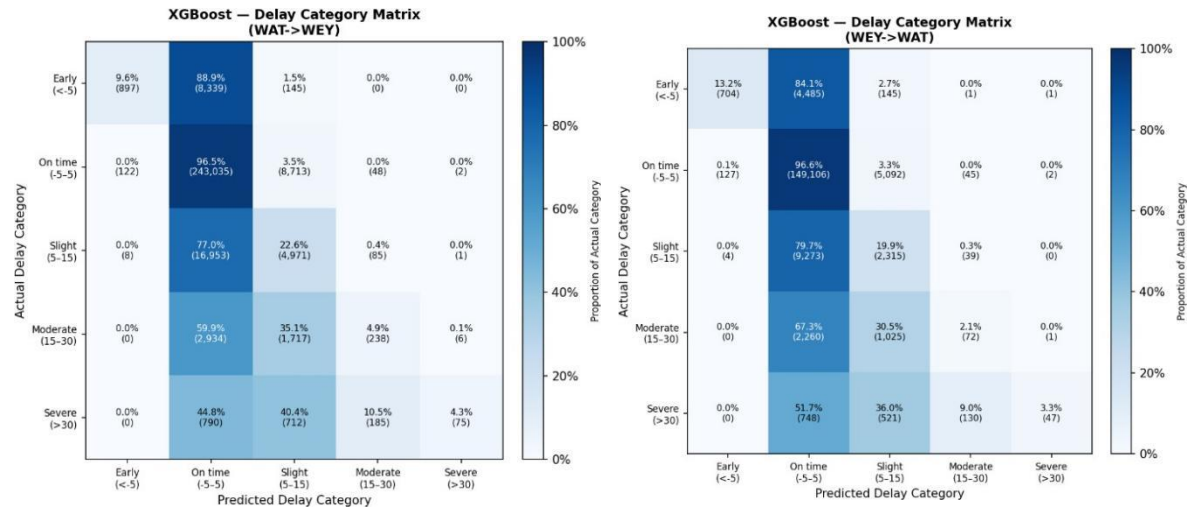


Fig6.3: Delay Category Matrix (XGBoost)

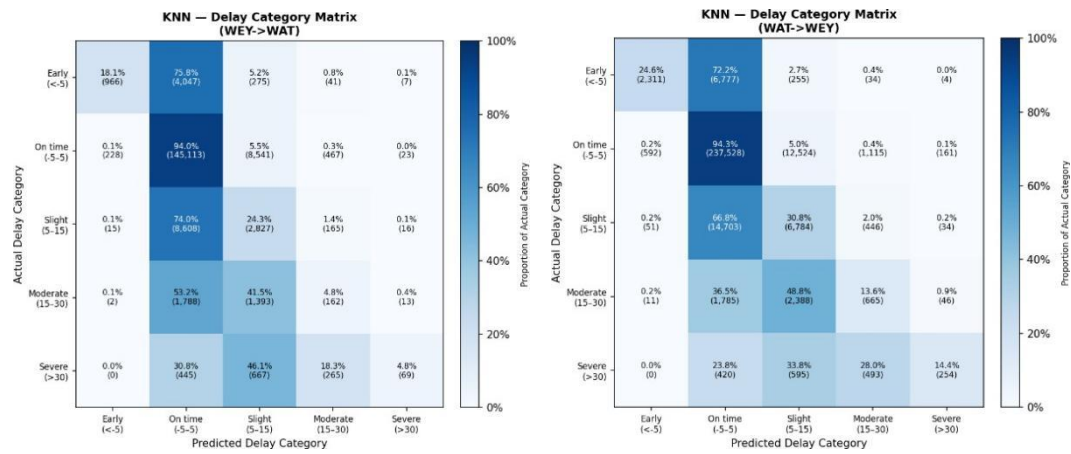


Fig6.4: Delay Category Matrix (KNN)

4.7.6 Model Selection and Justification

Across both directions and all three-evaluation metrics, Random Forest consistently delivers the strongest performance. The model files produced were large (approximately 578,589 KB in Random Forest combining both directions models). XGBoost, by contrast, produced substantially smaller model files. Nevertheless, despite the larger file size, the time taken to load and generate a prediction within the chatbot was negligible from a user perspective. Hence, given its superior accuracy across all metrics and both directions, Random Forest was therefore selected as the deployed model for the chatbot system.

4.7.7 Integration into the Chatbot system

To generate a prediction, the chatbot first collects several information from the user such as the direction or the train the user is, the station the user is travelling from, the destination, the planned arrival time at the destination, the current station the train is currently at, how many minutes the train has been delayed so far or current delay in minutes, etc.

Using these inputs, all twelve model features such as `planned_dep_min`, `day_of_week`, `month`, `is_weekend`, `is_peak`, `hour`, `station_num`, `journey_progress`, `stations_remaining`, `current_delay`, `destination_num`, `stops_to_dest` is computed.

The system determines the travel direction and loads the appropriate trained random forest model; one model for London Waterloo to Weymouth services and another for Weymouth to London Waterloo services. The selected model predicts the expected additional delay in minutes based on the generated feature vector. Finally, the chatbot estimates the updated arrival time.

4.7.8 Conversation Control

For task 2, six slots were collected from the user; direction, origin, destination, planned_arrival_time, current_station and current delay. Each slot is subject to its own validation rules to ensure the data passed to the prediction models is well-formed. Direction must be either from London Waterloo to Weymouth or Weymouth to London Waterloo. The origin and destination must correspond to recognized stations on the route. Planned arrival time must be entered in HH:MM format. Current station must also be a valid station on the line and current delay must be a whole number falling between 0 and 180 minutes.

If a user provides a value that fails validation, an error message is displayed explaining the issue and the system asks the user to enter valid data for the same slot rather than advancing to the next one. This ensures no invalid data reaches the backend.

The system also supports passive extraction. It can gather certain slot values directly from natural language without requiring the user to answer structured prompts. For example, a user saying “Weymouth to London Waterloo” allows the system to automatically derive both the origin and destination, and the direction without needing to ask for each value explicitly.

4.8 Task 3: Railway Disruption Expert System

4.8.1. What Is An Expert System?

An expert system is an artificial intelligence system that uses specialist domain knowledge to make decisions or provide advice in a way that resembles a human expert. Instead of learning only from data like a machine-learning model, an expert system normally relies on known facts, rules, procedures, and reasoning logic.

In this project, the expert knowledge comes from railway contingency plans and station disruption plans. The system uses this knowledge to advise users during railway incidents.

The main parts of an expert system are:

Component	Meaning In This Project
Knowledge base	Stores railway contingency plans, station disruption plans, passenger information, signaller information, and staff instructions.
Reasoning engine	Selects the correct plan or guidance based on the incident type, location, blockage type, and requested advice.
User interface	The web chatbot where the user asks questions and receives answers.
Working memory/session state	Stores the current conversation details, such as event type, station, line section, and requested guidance.
Explanation/output layer	Converts the selected expert knowledge into a readable chatbot response.

4.8.2 Introduction

Task 3 focuses on building an expert-system chatbot that can support railway disruption decision-making. The system is designed to help users obtain relevant contingency advice for two major disruption types:

- **Line blockage**, where a railway section between two stations or locations is partially or fully blocked.
- **Station disruption**, where a specific station is affected by an operational, safety, communication, access, or passenger-management issue.

The expert-system component is part of a wider railway chatbot application that also supports ticket search and delay or arrival prediction. Task 3 is different from the other tasks because it is not mainly about prediction or ticket lookup. Instead, it uses a structured knowledge base and a reasoning process to select the most appropriate operational guidance from railway contingency documents.

The chatbot guides the user through a short decision-making conversation. It asks what type of disruption is being handled, asks for the affected location, asks for blockage severity when relevant, and then asks what type of advice is needed. The final answer is formatted as practical user-facing guidance rather than a technical database result.

4.8.3 Expert System Architecture

Expert System Architecture

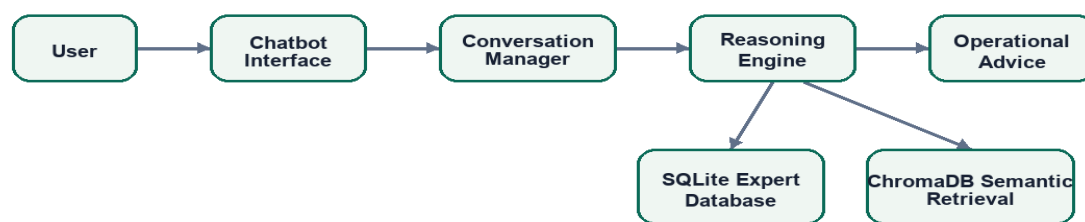


Fig 7: Expert- System Architecture

This diagram shows that the user interacts with the chatbot interface, while the actual expert reasoning happens in the backend. The conversation manager collects the required information, the reasoning engine searches the expert knowledge base, and the final operational advice is returned to the user.

4.8.4. Knowledge Base

A knowledge base is the stored collection of facts, rules, procedures, and expert guidance used by an expert system. In this project, the knowledge base contains railway operational guidance extracted from contingency planning documents.

The Task 3 knowledge base is built from:

- SWR line blockage contingency plan
- SWR station disruption plan
- Extracted text files stored in `knowledge\_base\_task3`.
- Structured records inserted into the SQLite expert database.
- Semantic rule cards stored in ChromaDB for retrieval fallback.

Knowledge Categories

The knowledge base contains two main categories.

1. Line Blockage Plans

Line blockage plans describe what should happen when a section of railway is blocked. These plans are organised by route section and blockage condition. The stored guidance includes:

- **Principles of Service Alteration:** how services should be cancelled, diverted, terminated, or started back.
- **Alternative Passenger Journey:** replacement buses, local buses, taxis, or alternative travel arrangements.
- **Information for Signallers:** operational notes for signallers.
- **Additional Information for Station Staff:** instructions for staff at affected stations.
- **Passenger Information:** passenger-facing travel advice.

2. Station Disruption Plans

Station disruption plans describe how staff should manage disruption at a station. The stored guidance includes:

- local station communications;
- communication with Control;
- practical operation of the station;
- alternative transport information;
- station-specific tips and support information.

4.8.4.1. Database Used And Why

The main expert-system database used for Task 3 is SQLite.

Item	Value
Database file	`task3_expert.db`
Configuration variable	`TASK3_EXPERT_DB_PATH`
Main Python module using it	`app/expert_system.py`
Ingestion script	`scripts/ingest_task3_expert_kb.py`

ChromaDB Semantic Retrieval

The system also uses **ChromaDB** as a persistent vector database.

Item	Value
Vector database path	`chroma_db`
Task 3 collection	`swr_contingency_expert_kb`
Main Python module using it	`app/task3_rag.py`

ChromaDB is used for semantic retrieval. This means that if an exact SQL match is difficult, the system can still search for operational rules that are semantically similar to the user request. This provides a useful fallback for natural-language queries.

4.8.5 Database Relationship Diagram

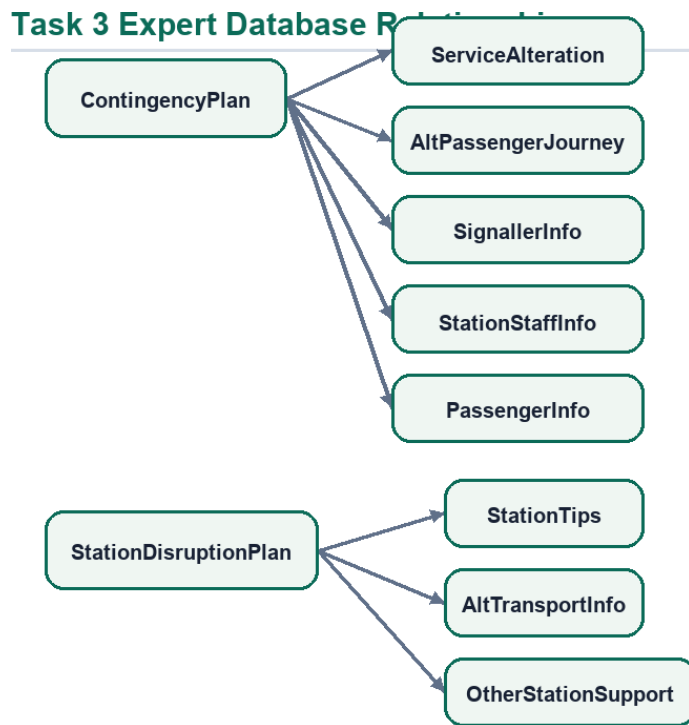


Fig 8: Database Relation

The diagram shows the main tables used by the Task 3 expert system. `ContingencyPlan` is linked to different advice tables for line blockage plans. `StationDisruptionPlan` is linked to station-specific disruption guidance.

4.8.6. Reasoning Engine Design

The reasoning engine is implemented mainly in `app/expert\_system.py`. Its job is to take the information collected by the chatbot and decide which expert guidance should be returned.

The reasoning engine uses a hybrid approach:

1. **Structured SQL matching** against `task3\_expert.db`.
2. **Semantic retrieval** from ChromaDB when useful.
3. **Role-specific response shaping** so the answer only includes the requested guidance.

Reasoning Method

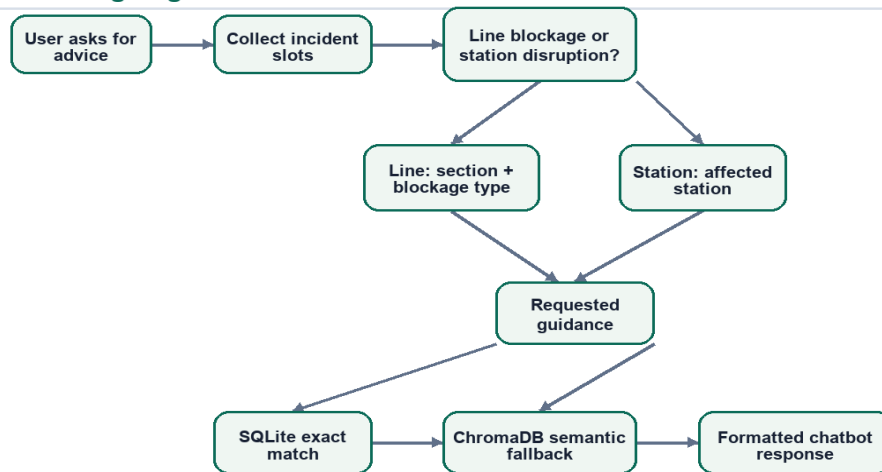
The reasoning process works as follows:

1. The chatbot collects slots from the user.
2. The system identifies the event type:
 - line blockage;

- station disruption.
- For a line blockage, the system uses:
 - first station or location,
 - last station or location,
 - blockage type, such as both lines blocked or one line blocked.
 - For a station disruption, the system uses:
 - affected station name.
 - The system retrieves relevant advice from SQLite.
 - ChromaDB is searched for semantic support or fallback guidance.
 - The final response is formatted as natural chatbot advice.

Reasoning Flow Diagram

Reasoning Engine Flow



Generated diagram for the Task 3 expert system report

Fig 9 : Reasoning FLOW Diagram

4.8.7 Chatbot Conversation Flow

The guided conversation is managed in `app/conversation.py`. This module controls the chatbot state, stores slots, asks missing questions, and formats final replies.

For Task 3, the chatbot follows this flow:

- Ask whether the incident is a line blockage or station disruption.
- If line blockage:
 - ask for the affected two stations or locations;
 - ask whether the blockage is partial or full;

- ask which guidance is required.
3. If station disruption:
- ask which station is affected;
 - ask which guidance is required.
4. Call the expert system.

Return the selected contingency or station disruption guidance.

4.8.8 Chatbot Flow Diagram

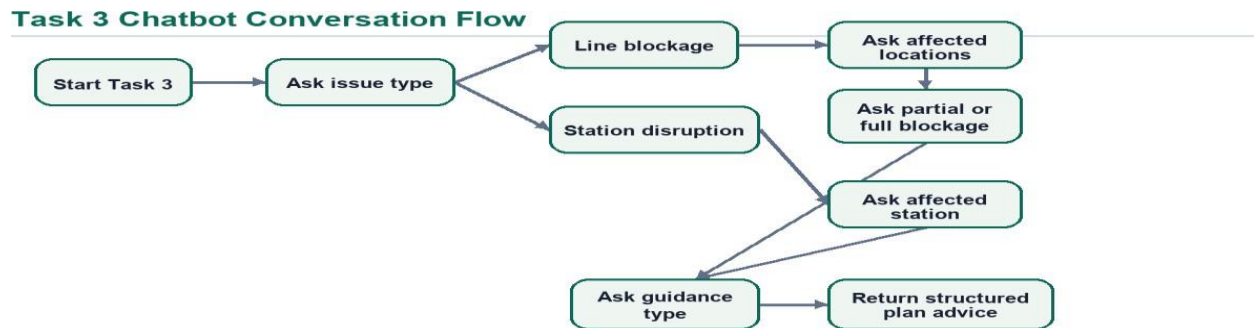


Fig 10 : Chatbot Conversation Workflow

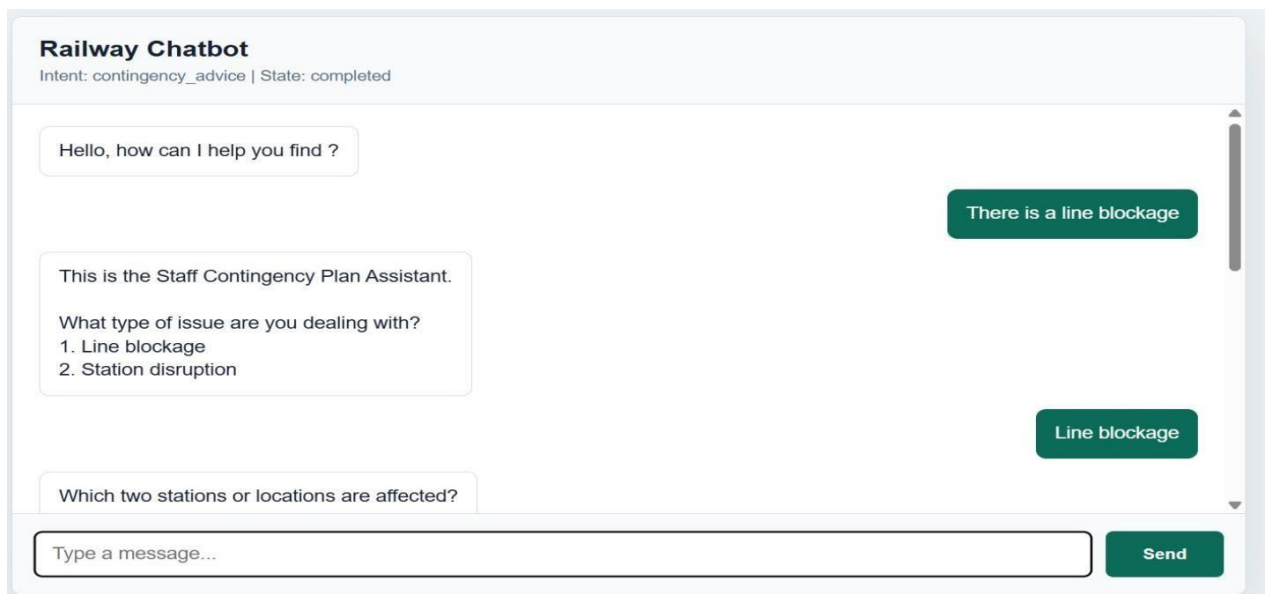


Fig 11 Conversation between the staff and Chatbot

Railway Chatbot

Intent: contingency_advice | State: completed

Line blockage

Which two stations or locations are affected?

Havant - Portcreek Junction Portsmouth Direct

Is the blockage partial (one line blocked) or full (both lines blocked)?

One line blocked

Do you need station staff advice, signaller advice, passenger advice, service alterations, alternative routes, or all available guidance?

Send

Fig 11.1 : Staff asking the chatbot about the disruption caused

Railway Chatbot

Intent: contingency_advice | State: completed

Do you need station staff advice, signaller advice, passenger advice, service alterations, alternative routes, or all available guidance?

Station staff advice

Contingency Plan 24.2
Location: Havant - Portcreek Junction
Condition: One Line Blocked

Additional Information for Station Staff:
SWR bus replacement to Portsmouth Harbour Fast bus calling at Portsmouth & Southsea. Slow bus calling all stations, including Cosham at Havant.

Would you like to see additional contingency plans for this incident?

Send

Fig 11.2 Chatbot giving the alternate plan

5. Implementation

5.1 Task 1

Task 1 is executed through a FastAPI backend that exposes a single `/api/chat` endpoint, which receives the user’s message and session ID from the browser interface. When a request arrives, the

backend passes the message to the conversation manager, which runs the intent detector and rule-based entity extractors to identify stations, dates, times, railcards, and journey preferences. The conversation manager then checks which slots are still missing and returns the next appropriate question, ensuring the dialogue progresses logically. All session data is stored in SQLite so the chatbot remembers previous turns. Once all required slots are filled, the backend either calls the National Rail API or triggers the mock ticket generator to produce journey results. The final ticket summary is then returned to the frontend, where it is displayed to the user. This execution pipeline allows the chatbot to run continuously, handle multi-turn conversations, and generate ticket results in real time.

5.2 Task 2

5.2.1 Delay Predictor (`delay_predictor.py`)

`predict_delay_ml()` converts the extracted slot values into formats suitable for prediction. Station names are mapped into their corresponding CRS codes, the planned arrival time is parsed into a datetime object, and the current delay is converted into an integer. The travel direction is either provided directly through the slots or inferred by comparing the relative positions of the origin and destination within the known station sequence. Finally, Random Forest model is loaded using `joblib` depending on the direction of the journey.

A twelve-feature array is then constructed to match the training feature set, covering temporal features such as hour, day, month, peak/weekend flags, positional features such as station index, journey progress, stations remaining and current delay. The array is then passed to `model.predict()`, returning the additional delay expected between the current station and destination. A dictionary containing the prediction, expected arrival time, risk level, recommendation, and supporting reasons is also returned.

5.2.2 Conversation Manager (`conversation_manager.py`)

The manager maintains conversational context between turns through `ChatStorage`. For each user input, the intent is detected if it has not already been established, and `fill_slot_from_direct_answer()` attempts to extract and populate the current slot from the user's input. Each slot is validated using rules; station names are checked against the CRS map, times must match HH:MM format, and delays must be a whole number between 0 and 180. If validation fails, the error is stored in `_validation_error` and the user is prompted again for the same slot; otherwise, the flow advances to the next required slot. Once all six slots have been successfully

completed, `_complete_flow()` invokes the prediction model and returns the formatted prediction result.

5.2.3 Arrival Time Estimation

Expected arrival is calculated as:

$$\text{expected_arrival} = \text{planned_arrival} + (\text{current_delay} + \text{ml_additional_delay})$$

The planned arrival and current delay are provided by the user. The ML model predicts any further delay expected before reaching the destination. For example, a train due at 10:30, already 10 minutes late, with a predicted additional 7 minutes, would return an estimated arrival of 10:47.

5.3 Task -3

Area	Reference
Chatbot UI	`app/static/index.html`
API entry point	`app/main.py`
Conversation flow	`app/conversation.py`
Expert reasoning engine	`app/expert_system.py`
ChromaDB retrieval	`app/task3_rag.py`
Task 3 ingestion	`scripts/ingest_task3_expert_kb.py`
Structured expert database	`task3_expert.db`
Extracted knowledge files	`knowledge_base_task3`

6. Testing

Testing was carried out to check that the railway chatbot works correctly across all three coursework tasks. The purpose of testing was to confirm that the system can understand user requests, collect missing information through conversation, call the correct backend logic, and return useful responses. Testing also helped confirm that invalid or incomplete inputs are handled before they reach the prediction model or expert-system reasoning engine.

The testing process was divided into unit testing, integration testing, system testing, and usability testing.

6.1 Unit Testing

Unit testing focused on checking individual parts of the system separately. Each component was tested to make sure it performed its own responsibility correctly before being used as part of the full chatbot.

The main areas tested were:

Intent detection: The chatbot was tested with different user messages to confirm whether it correctly identified ticket search, delay prediction, arrival prediction, or contingency advice.

Slot extraction: The system was checked to ensure that stations, dates, times, delay minutes, disruption locations, blockage type, and requested advice categories were extracted correctly.

Ticket search logic: The ticket workflow was tested to confirm that it collects origin, destination, travel date/time, and single or return preference.

Task 2 validation logic: The arrival prediction component was tested to ensure missing delay minutes, unrecognised stations, invalid station names, and missing model data are handled safely.

Task 3 reasoning logic: The expert system was tested to confirm that line blockage and station disruption queries are routed to the correct knowledge-base lookup.

These unit tests helped verify that each function works correctly before the complete chatbot flow is tested.

6.2 Integration Testing

Integration testing checked whether the different components work correctly together. This was important because the chatbot depends on several connected parts: the frontend interface, FastAPI backend, conversation manager, natural language processing, database storage, prediction logic, and expert-system reasoning.

The integration testing checked that:

- the web interface sends messages correctly to `/api/chat`,
- FastAPI receives the request and returns a valid response,
- the conversation manager stores and updates session state,
- extracted slots are passed correctly to the ticket, prediction, or expert-system modules,
- SQLite is used correctly for session storage and Task 3 expert knowledge lookup,
- ChromaDB retrieval works as a fallback for Task 3 semantic matching,
- the final chatbot response is returned to the browser without exposing debug or source information.

Integration testing confirmed that the chatbot works as a combined system rather than as isolated files.

6.3 System Testing

System testing tested the complete chatbot from the user's point of view. The aim was to check whether the full system behaves correctly for realistic user queries across Task 1, Task 2, and Task 3.

6.3.1 Task 1

Task 1 focuses on finding the cheapest train ticket for a user's journey. The chatbot was tested with journey-related questions such as asking for a ticket from one station to another at a specific time. The system was checked to ensure that it asks for missing information such as:

- origin station,
- destination station,

- travel date and time;
- single or return ticket preference.

Expected behaviour: once all required information is collected, the chatbot should return a structured ticket response showing the route, ticket type, price or availability status, provider, and booking link.

6.3.2 Task 2

Task 2 focuses on delay and arrival-time prediction. Before any input is passed to the prediction model, the chatbot performs a series of input validation checks to ensure no invalid data gets entered into the model. The following scenarios are handled explicitly:

- missing delay minutes,
- delay minutes entered in an invalid format,
- missing current station,
- unrecognised station names,
- missing destination,
- unclear direction of travel,
- missing or unavailable trained model file,
- missing service context,
- unrealistic predicted delay values.

These validation steps ensure that malformed or out-of-range inputs are caught at the conversational layer, preventing wrong predictions and maintaining a reliable user experience.

The system was also tested with valid delay examples, such as a train delayed by a specific number of minutes at a known station and travelling to London Waterloo or Weymouth. The expected result is that the chatbot returns a predicted arrival time, final predicted delay, current station, destination, and a short explanation of how the prediction was made.

6.3.3 Task 3

Task 3 focuses on the expert-system chatbot for railway disruption advice. The system was tested with both main disruption types:

- line blockage,
- station disruption.

For line blockage, the chatbot was tested to confirm that it asks for:

- whether the user needs line blockage or station disruption advice;
- the two affected stations or locations;
- whether the blockage is partial or full;
- the type of guidance required.

For station disruption, the chatbot was tested to confirm that it asks for:

- the affected station,
- the type of guidance required.

Example Task 3 test cases included:

- passenger advice for both lines blocked between Woking and Guildford,
- station staff advice for disruption at Bournemouth,
- signaller guidance for a line blockage,
- service alteration guidance for a blocked route,
- all available guidance for a disruption.

Expected behaviour: the chatbot should return focused operational guidance from the expert knowledge base. It should not show technical source paths, ChromaDB retrieval details, debug information, or unnecessary explanation text.

Railway Chatbot
Intent: delay_prediction | State: collecting_origin

Hello, how can I help you today?

i want to predict a delay

Which direction are you travelling? Reply 'WAT2WEY' for London Waterloo → Weymouth, or 'WEY2WAT' for Weymouth → London Waterloo.

wat2weyy

Sorry, I didn't recognise 'wat2weyy' as a valid direction. Please reply 'WAT2WEY' for London Waterloo → Weymouth, or 'WEY2WAT' for Weymouth → London Waterloo.

wat2wey

What station are you travelling from?

Type a message...

Send

Figure12: Invalid direction error

Railway Chatbot
Intent: delay_prediction | State: collecting_origin

Hello, how can I help you today?

i want to predict a delay

Which direction are you travelling? Reply 'WAT2WEY' for London Waterloo → Weymouth, or 'WEY2WAT' for Weymouth → London Waterloo.

wat2weyy

Sorry, I didn't recognise 'wat2weyy' as a valid direction. Please reply 'WAT2WEY' for London Waterloo → Weymouth, or 'WEY2WAT' for Weymouth → London Waterloo.

wat2wey

What station are you travelling from?

ktn

Sorry, I don't recognise 'ktn' as a station on this route. Please enter a station name such as London Waterloo, Bournemouth, or Weymouth.

Type a message...

Send

Figure 12.1: Invalid direction of train and invalid origin station error

What is your destination station?

jorpati

Sorry, I don't recognise 'jorpati' as a station on this route. Please enter a station name such as London Waterloo, Bournemouth, or Weymouth.

Weymouth

What is your planned arrival time at your destination? Please use HH:MM format (e.g. 10:30)

Type a message...

Send

Figure 12.2: Invalid destination station error

What is your planned arrival time at your destination? Please use HH:MM format (e.g. 10:30)

any

Sorry, 'any' doesn't look like a valid time. Please use HH:MM format, for example 10:30 or 08:15.

120000

Sorry, '120000' doesn't look like a valid time. Please use HH:MM format, for example 10:30 or 08:15.

12:901

Sorry, '12:901' doesn't look like a valid time. Please use HH:MM format, for example 10:30 or 08:15.

any1

Sorry, 'any1' doesn't look like a valid time. Please use HH:MM format, for example 10:30 or 08:15.

Type a message...

Send

Figure 12.3: Invalid planned arrival time data entry error

7. Evaluation and Discussion

The railway chatbot was evaluated as a combined system covering all three coursework tasks: ticket search, delay or arrival prediction, and disruption contingency advice. Overall, the system successfully demonstrates how different AI techniques can be combined in one chatbot interface. Task 1 uses conversational slot collection to gather journey details for ticket search. Task 2 uses validation and prediction logic to estimate delay or arrival outcomes. Task 3 uses an expert-system approach with a structured knowledge base, SQLite matching, and ChromaDB semantic retrieval to provide railway disruption guidance.

A key strength of the chatbot is its guided conversation flow. Instead of expecting the user to provide every detail in one sentence, the chatbot asks follow-up questions when required information is missing. This improves reliability because the system collects the correct inputs before calling the ticket search service, prediction model, or expert-system reasoning engine. This is especially important for Task 2, where invalid prediction inputs could lead to unreliable results, and for Task 3, where missing disruption details could cause the wrong contingency plan to be selected.

The chatbot also separates user-facing answers from technical backend details. For example, Task 3 responses are presented as practical operational guidance rather than showing raw source files, database results, or ChromaDB retrieval information. This makes the system easier for normal users to understand. At the same time, the backend still uses structured data and retrieval logic to support accurate matching.

However, there are some limitations. The ticket search depends on the availability of the external ticket API or service integration. The prediction task depends on the quality and availability of the trained model and the input data provided by the user. The expert-system task depends on the quality of extracted contingency documents and station disruption plans. If station names are misspelled or the user gives vague information, the system may need extra clarification or may return a less accurate result.

Overall, the chatbot meets the main coursework aim because it provides a working multi-task railway assistant. It demonstrates natural-language interaction, input validation, prediction support, expert-system reasoning, database use, and a simple browser-based user interface.

8. Conclusion and Summary

This project developed a railway chatbot that combines three main functions: finding train ticket information, predicting delay or arrival outcomes, and providing expert-system contingency advice for railway disruption scenarios. The system is built using a FastAPI backend, a simple web chatbot interface, SQLite storage, prediction logic, and a Task 3 expert knowledge base supported by ChromaDB retrieval.

The final chatbot shows that a conversational interface can make complex railway information easier to access. For Task 1, the chatbot collects journey details and returns ticket-related information. For Task 2, it validates user inputs before making predictions, helping to prevent invalid data from entering the model. For Task 3, it acts as a staff contingency plan assistant by asking guided questions and returning focused disruption guidance for line blockages and station disruptions.

The most important achievement of the system is that it combines multiple intelligent behaviours in one application. It does not only answer simple questions; it remembers conversation state, asks follow-up questions, validates inputs, searches structured data, uses expert-system reasoning, and produces user-friendly responses.

In summary, the project successfully demonstrates how AI and rule-based techniques can be applied to railway customer and staff support. Future improvements could include stronger station-name normalisation, richer frontend controls, live operational data integration, and more automated testing.

9. Reference

- [1] FastAPI. (2026). FastAPI documentation. Available at: <https://fastapi.tiangolo.com/>
- [2] Uvicorn. (2026). Uvicorn ASGI server documentation. Available at: <https://www.uvicorn.org/>
- [3] Python Software Foundation. (2026). Python documentation.
Available at: <https://docs.python.org/3/>
- [4] SQLite. (2026). SQLite documentation. Available at: <https://www.sqlite.org/docs.html>
- [5] Chroma. (2026). ChromaDB documentation. Available at: <https://docs.trychroma.com/>
- [6] Pandas. (2026). Pandas documentation. Available at: <https://pandas.pydata.org/docs/>
- [7] Scikit-learn. (2026). Scikit-learn documentation.
Available at: <https://scikit-learn.org/stable/documentation.html>
- [8] Joblib. (2026). Joblib documentation. Available at: <https://joblib.readthedocs.io/>
- [9] South Western Railway. (n.d.). Railway contingency and station disruption planning documents. Internal/coursework source material used for Task 3 knowledge-base extraction.
- [9] National Rail. (2026). Online Journey Planner data feeds. Available at: <https://www.nationalrail.co.uk/developers/online-journey-planner-data-feeds/>